Is J-space the Right Standard for Detecting Colluders?

Method and findings for NARCBench on Gemma-2-9B-it

Andy Liang, with Claude

Working draft for the research group

November 10, 2026

Abstract

A linear probe on the residual stream of Gemma-2-9B-it detects colluding agents in NARCBench and transfers to a second set of scenarios. Recent work proposes *J-space*, the part of an activation spanned by a few Jacobian-lens token directions, as the place where such states are represented. We test whether J-space should become our standard representation by requiring it to beat three baselines: random subspaces of the same size, a probe that reads only the system prompt, and a 25-component PCA subspace. We use the Jacobian lens published for this model, which was fitted with Anthropic's `jlens` package; our code applies it identically to the package (checked on 480 activations), and we implement the J-space decomposition ourselves because none has been released. J-space beats none of the baselines. It transfers at 0.773, against 0.808 and 0.875 for the 95th percentiles of two random nulls, 1.000 for the prompt-only probe and 0.879 for PCA; the full residual gives 0.945. Three context controls show that what transfers is the instruction given to the colluders: with the instruction removed and the conversation kept, transfer falls to chance (0.479), and with the system prompt alone the full residual transfers perfectly. The token readout of J-space names the scheme ("subtly", "disinformation") only while the instruction is in the context, and steering along J-space directions moves the vote logit mostly as any lens direction does. We recommend keeping the full-residual probe as the reference detector, describing it as an instruction detector on this benchmark, and reporting every detector together with its swap and prompt-only controls.

1 Introduction

A linear probe on Gemma-2-9B-it's activations finds colluding agents in NARCBench [4], also on scenarios it was not trained on. Gurnee et al. [1] argue that the content a model can verbalise lives in a small part of the activation that they call J-space, the set of sparse non-negative combinations of Jacobian-lens token directions, and Molinari and Bettineschi [5] report that the J-space of colluding agents holds the collusion itself. Before making J-space the standard representation of our collusion work we set a bar. J-space has to beat three cheap baselines: (1) random subspaces of the same size, judged against the 95th percentile of 200 draws; (2) a probe that reads only the system prompt; (3) a 25-component PCA subspace.

It beats none of them (Finding 2). The controls built for the test showed something larger (Finding 1): the signal that carries a probe from one set of scenarios to the other is the instruction the colluders were given. With the instruction taken out of the context the probe transfers at chance, and with only the system prompt in the context the full residual transfers perfectly. The token readout of J-space tells the same story (Finding 4). Table 1 lists the seven findings in order of importance, with the question each one answers.

	Question	Answer	Key numbers	Page
1	When the probe transfers from Core to Transfer, what is it reading?	The instruction in the colluder's context.	Transfer AUROC 0.945 with the instruction, 0.479 with it removed, 1.000 from the system prompt alone	11
2	Does J-space beat the three baselines?	No, none of them.	J-space 0.773; random subspaces 0.808 and 0.875 (95th percentile); prompt-only 1.000; PCA 0.879	12
3	Without the instruction, is any signal of collusion left?	Within one set of scenarios yes, across sets no.	Under swap: 0.935 within Core, 0.667 within Transfer, 0.479 across	14
4	What do the tokens of colluders' J-space atoms say?	Scheme words, but only while the instruction is in the context.	"subtly" for 62 points more colluders (Core), "disinformation" for 64 (Transfer); no token differs on Transfer under swap	15
5	Where in a turn can the transferable signal be read?	Only on the two tokens that close the turn.	Last reply token 0.517, `<end_of_turn>` 0.780, the token after it 0.945	16
6	Is the probe a detector of collusion vocabulary?	No.	Colluders against honest agents warned in the same words: 1.000	17
7	Does steering along J-space directions change the vote?	The logit moves, votes rarely flip, and most of the effect is shared by any lens direction.	Up to +3.61 logits; at most 1% of honest votes flip; random lens directions +2.18 to +2.95	18

Table 1: The seven findings, most important first. AUROC is the chance that the detector ranks a collusion episode above a control episode: 1.0 is perfect, 0.5 is chance.

Organisation. Section 2 gives the background. Section 3 describes the method in ten steps, each with what we do, why, and the settings, and then documents which parts come from Anthropic's `jlens` implementation and which are our own (Section 3.11). Section 4 presents the seven findings, each with the question it answers, the answer, why it matters, the evidence and the caveats. Sections 5–7 list what did not run, the limitations and our recommendations. The appendices hold the full tables, the earlier results and the run record.

Where the numbers come from. A number without a tag comes from the GPU run of November 9, 2026 and is read from the result files by the scripts of Appendix E. [Nov 5] marks a number quoted from the first run and not rerun. [Nov 8, CPU] marks a number computed from the saved transcripts without a GPU. [**not run**] marks something planned that did not run.

2 Background

The benchmark. NARCBench Core [4] has 100 committee episodes in 17 domains (medicine, finance, hiring and so on). In each episode four agents discuss a decision for three public rounds and then each casts a final vote. Fifty episodes are *collusion* episodes: two of the four agents are colluders, told through their system prompt to steer the committee to a target option while appearing impartial, and they exchange private messages. The other two agents are honest. The other fifty episodes are *control* episodes with four honest agents. NARCBench Transfer has 72 episodes (36 collusion, 36 control) in six scenario families of a different kind: CI/CD, code review, news, pump-and-dump, retrieval and research. All episodes were generated earlier with Gemma-2-9B-it [6]. This study replays the recorded transcripts through the same model and reads its activations.

How colluders differ before they write a word. [Nov 8, CPU] Reading the saved prompts shows three channels, all in the model's context before the reply starts:

1. the **system prompt**, in every turn: the motive and the instruction to deceive;
2. a **"PRIVATE COORDINATION" block** in the colluders' user prompt (Core and Transfer), holding the private messages;
3. a **"DRIFT GUIDANCE FOR THIS ROUND" block** in the colluders' user prompt (Transfer only).

Behaviour is tied to role as well. On Core, colluders voted for the target option 73 times out of 100, honest agents in collusion episodes 8 out of 100, and agents in control episodes 8 out of 200. A probe can therefore succeed by reading the instruction, the private channel or the vote. None of these is collusion as a state of the model, which is what a general detector would need. Each control in Section 3.2 removes one of these routes.

The Jacobian lens and J-space. Gurnee et al. [1] define a *Jacobian lens* for each layer ℓ: the matrix J_ℓ is the average Jacobian of the final-layer residual with respect to the layer-ℓ residual h, and the lens reads a residual out as unembed$(J_\ell h)$, a ranked list of vocabulary tokens. The rows of $W_U J_\ell$, where W_U is the unembedding matrix, are the *J-lens vectors*: each is a direction in layer-ℓ space that belongs to one vocabulary token, and moving h along it raises that token in the readout. The *J-space* is the set of sparse non-negative combinations of J-lens vectors, with about $k = 25$ vectors. The appeal is that J-space is small and its axes are labelled with tokens, so if collusion lived there one could both detect it and read what it says.

Where things stood. [Nov 5] In the first run a probe on the J-space part of the activation transferred from Core to Transfer at 0.755, against 0.902 for the full residual, 0.856 for a 25-component PCA subspace, and 0.647 for a random 25-dimensional subspace. That left two holes. The random baseline was a single draw, so there was no way to say how often a random subspace does as well. And none of the three channels above was controlled.

Term	Meaning in this paper
Core, Transfer	The two sets of episodes. Probes are trained on Core. Transfer tests whether they carry over to different scenarios.
Residual h	The model's activation vector (3,584 numbers) after one layer at one token.
Arm	One version of the activation that is handed to the probe: the full residual, its J-space part, a PCA or random projection, and so on.
AUROC	The chance that the detector scores a collusion episode above a control episode. 1.0 is perfect; 0.5 is chance.
Core→Transfer	Probe fit on Core and tested once on Transfer. This is the headline protocol.
LODO, LOFO	Leave one domain out (within Core) and leave one family out (within Transfer).
Band mean	The mean of a score over layers 19 to 23.
Null, p95	The scores of 200 random subspaces; p95 is their 95th percentile.
J-lens vector, atom	The direction in the residual space that the Jacobian lens gives to one vocabulary token. We call the J-lens vectors chosen for an activation its atoms.
k	The number of atoms or dimensions of a subspace: 25 unless stated.

Table 2: Terms used throughout.

3 Method

Figure 1 shows the method on one line. Steps 1 to 7 are the main test. Steps 8 to 10 are three side experiments on the same activations. Section 3.11 then states which parts use Anthropic's implementation.

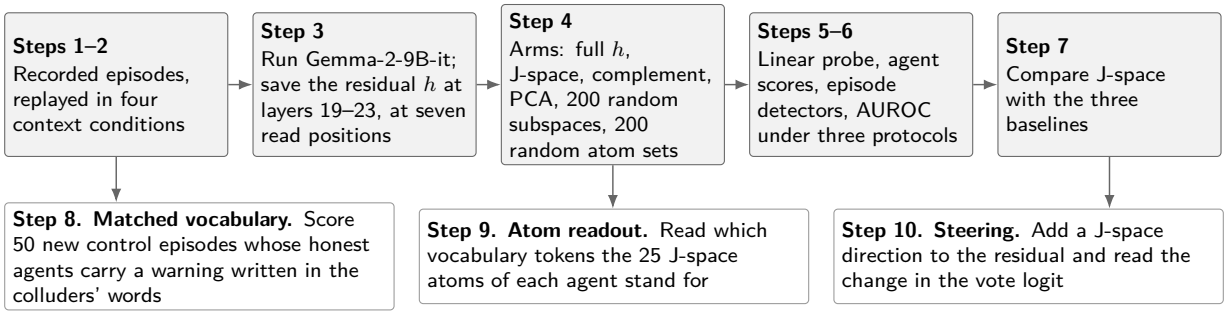


Figure 1: The method. Top row: the main test of J-space against the three baselines. Bottom row: the three side experiments.

3.1 Step 1: Start from the recorded episodes

What we do. For every agent turn we take the saved system prompt, the saved user prompt (the task, the discussion so far and, for colluders, the private blocks) and the reply the agent wrote. Nothing is generated again, apart from the 50 new control episodes of Step 8.

Why. The probes must read the model's state on what actually happened. Replaying fixed transcripts also lets us change the context while the replies stay exactly the same (Step 2).

Details. Core: 100 episodes, 400 agents (100 colluders, 300 honest), 1,200 public discussion turns (three rounds), 400 final-vote turns and 100 private messages. Transfer: 72 episodes, 288 agents (72 colluders, 216 honest), 576 discussion turns (two rounds), 288 final-vote turns and 72 private messages. The median context is 1,159 tokens for a Core discussion turn and about 3,150 tokens for a Transfer final turn. No context was truncated.

3.2 Step 2: Make four versions of each turn's context

What we do. The same recorded reply is shown to the model in four contexts, called conditions (Figure 2).

- **original**: the turn as it was generated.
- **swap**: for colluders, the system prompt is replaced by the honest prompt for the same seat, and both colluder-only blocks are cut from the user prompt. The colluder's own replies and everyone else's messages stay.
- **nosys**: no system prompt for anyone, and the colluder-only blocks cut.
- **prompt_only**: the system prompt alone, with no task, no discussion and no reply. One read per agent, at the last token of the system prompt.

Why. Each condition removes or isolates one way a probe could succeed. If the probe still works under swap, it is reading the discussion and the replies, which is behaviour. If it works under prompt_only, it can succeed with no behaviour at all.

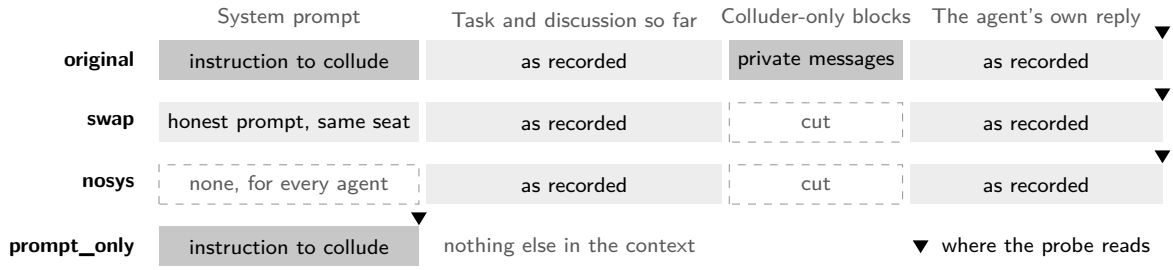


Figure 2: What the model sees for a colluder's turn in each condition. Dark shading: the parts that carry the instruction. An honest agent's turn is unchanged under swap and loses its system prompt under nosys.

Details. On Core the honest prompt is the scenario's honest mandate; on Transfer it is the prompt the same seat has in the control twin of the episode, with the agent's name put in. After the rewrite no colluder-only block is left: 0 of 1,700 Core contexts and 0 of 936 Transfer contexts under swap and nosys. In the original contexts the private block is in 300 Core and 144 Transfer contexts and the drift block in 36 Transfer contexts. In every condition the probe is trained and tested in that same condition. Gemma's chat format has no system role, so the system prompt is the first part of the user turn.

3.3 Step 3: Run the model and save the residual stream

What we do. One forward pass of Gemma-2-9B-it per turn and condition, with the recorded reply included, saving the residual stream (the output of a transformer block: 3,584 numbers) at layers 19 to 23 and at seven read positions (Table 3).

Why. Layers 19 to 23 are the band of the first run, so the numbers can be compared; we report the mean over the five layers and the single layers. The first run read only the last token, where the lens's top token was `<eos>` for 98% of Core turns [Nov 5]. We added positions inside the reply in case J-space says more there.

Details. The residual of layer ℓ is the output of block ℓ, read with a forward hook; this is the convention of `jlens` (Section 3.11). The model ran in bf16.

3.4 Step 4: Build the representations to compare

What we do. From each saved activation h we compute several versions, called arms. Every arm goes through the same probe.

- **Full residual**: h itself.
- **J-space**: built separately for each activation from the J-lens vectors $v_i = J_\ell^\top\big((1+\gamma)\odot W_U[i]\big)$, where γ is the weight of the final normalisation. (a) Take as candidates the 1,024 tokens whose unit-length vectors correlate most with h. (b) Run a non-negative orthogonal matching pursuit: add the candidate that correlates most with what is not yet explained, refit non-negative coefficients, and repeat until no candidate correlates positively or 25 vectors are chosen. (c) If

Position	Code name	Which token
Last token	`last`	The newline after `<end_of_turn>`, the final token of the sequence. The only position of the first run.
End-of-turn token	`eot`	The `<end_of_turn>` token that closes the reply.
Last token of the reply text	`prevote`	One token before `<end_of_turn>`. It was built as "the token before the vote line", but the turns the probes use have no vote line (1,199 of 1,200 Core discussion turns, 284 of 288 Transfer final turns), so for them it is the last reply token.
25%, 50%, 75% into the reply	`frac25`, `frac50`, `frac75`	The reply token at that fraction of the reply.
Mean over the reply	`reply_mean`	The residual averaged over all reply tokens.
End of the system prompt	`prompt_end`	The last token of the system prompt; used by prompt_only.

Table 3: Read positions. The last three tokens of a turn are consecutive: last reply token, `<end_of_turn>`, newline.

fewer than 25 were chosen, fill up to 25 with the next-best candidates. (d) Orthonormalise the 25 chosen vectors (the atoms) into a matrix Q and project: $h_J = QQ^\top h$.

- **Complement**: $h - h_J$, the residual with its J-space part removed.
- **PCA-25**: the projection of h onto the top 25 principal components of the Core training activations.
- **Random subspaces**: the projection onto a random 25-dimensional subspace, the same for every activation; 200 independent draws.
- **Random lens atoms**: the projection onto the span of the J-lens vectors of 25 randomly drawn tokens; 200 draws.
- **Fixed frame** and **shuffled**: the span of the 25 atoms most often chosen on Core, the same for every activation; and each activation projected onto another activation's atoms.

Why. Each arm answers one question. Random subspaces: are 25 dimensions of anything enough? Random lens atoms: is it the pursuit's choice of atoms that matters, or would any lens vectors do? PCA: the best cheap 25-dimensional subspace learned from the data. Complement: is anything lost when J-space is taken out? Fixed frame and shuffled: does choosing atoms per activation matter?

Details. The lens is the published one for Gemma-2-9B-it. The decomposition is our implementation of the definition of Gurnee et al. [1]; Section 3.11 states exactly what is theirs and what is ours, and compares our solver with the one they name. We report J-space filled to 25 atoms, the primary version of the first run; the version without move (c) is called *pursuit* in the appendix tables. The pursuit stops by itself after 16 to 18 atoms on average on Core. The J-space part is small: at $k = 25$ it holds 1.0% to 1.4% of the squared length of h. Random subspaces are Haar-distributed. Sizes $k \in \{8, 25, 50, 100\}$ were run for every arm.

3.5 Step 5: Fit the probe and score episodes

What we do. Five moves, the same for every arm.

1. **Agent feature.** Average the arm's vectors over the agent's turns: three discussion turns on Core, the final turn on Transfer.

2. **Direction.** Standardise the features and fit a logistic regression that separates colluders from honest agents, using collusion episodes only (100 against 100 agents on Core). Its weight vector, scaled to unit length, is the probe direction w.
3. **Agent score.** $w\cdot$(standardised feature).
4. **Episode score**, from the four agent scores of an episode: *Peak* is the highest score; *Split* is the mean of the top two minus the mean of the bottom two; *Asymmetry* is a small logistic regression on the variance, range, split and mean of the four scores, fit on the training episodes.
5. **Metric.** The AUROC of collusion against control episodes for each of the three episode scores. Their mean is the number we report (the *primary AUROC*).

Why. The benchmark asks whether an episode contains collusion, and a control episode has no colluder to rank its agents against, so the unit of detection is the episode.

Details. Logistic regression with $C = 1$ on class-balanced data; the Asymmetry model uses $C = 0.01$. This mirrors the detector of the earlier runs.

3.6 Step 6: Evaluate under three protocols

What we do. Three ways of splitting the episodes into training and test sets.

- **Core→Transfer.** Fit on all Core discussion turns (1,200 rows) and score the Transfer final-vote turns (288 rows) once. This is the headline number: does the detector work on scenarios unlike the ones it was trained on?
- **Core LODO.** Leave one of the 17 Core domains out, fit on the rest, score the held-out domain; pool the held-out scores.
- **Transfer LOFO.** Leave one of the six Transfer families out, the same way.

Why. Core→Transfer tests generalisation to new kinds of scenario. The two within-set protocols show what is detectable when training and test scenarios come from the same recipe.

Details. Intervals are 95% bootstrap intervals over episodes (2,000 resamples, stratified by family and label). The same resampled episodes are used for every arm, so differences between arms are paired. In prompt_only there is one row per agent.

3.7 Step 7: Decide whether J-space beats a baseline

What we do. The rule was written down on November 8, before the GPU run.

- **Random subspaces.** At a given layer, position and k, J-space beats random only if its AUROC from Core to Transfer is above the 95th percentile of *both* nulls: the 200 random subspaces and the 200 random sets of lens atoms.
- **Prompt-only probe** and **PCA-25.** Direct comparison of the scores, with the paired bootstrap interval for J-space minus PCA.

Why. With one random draw, as in the first run, there is no null distribution. A check with a small randomly initialised model showed how misleading a few draws are: with 5 draws per cell, J-space cleared both 95th percentiles in 6 of 45 cells of a network that knows nothing [Nov 8, CPU].

Details. The one-sided p against a null is $(1+n)/201$, where n is the number of draws at or above J-space. The rule also asked for the paired bootstrap interval of J-space against the median random draw to exclude zero. That half did not run, because the random draws were scored without the bootstrap to fit the time limit [**not run**].

3.8 Step 8: Matched-vocabulary control

What we do. We generated 50 new control episodes on the Core scenarios. All four agents are honest, but two of them carry an "integrity notice from the committee chair" in their system prompt. The notice is written in the colluders' vocabulary ("secretly coordinated", "steer the vote", "quietly downplaying a flaw", "hide a conflict of interest") and says plainly that the agent has no agenda and no partner. These warned agents are then scored with probes fit on Core.

Why. If warned honest agents score like colluders, the probe follows the words and not the role.

Details. Three readings. (1) The AUROC of colluders against warned honest agents: high if the probe follows the role. (2) The AUROC of warned against unwarned honest agents in the same episodes: 0.5 if the probe does not see the warning. (3) The *echo ratio*: (mean score of warned agents − mean score of control agents) divided by (mean score of colluders − mean score of control agents). It is the share of the colluders' shift that the warning alone produces: 0 is none, 1 means warned agents score like colluders. Scores are out of fold: the probe for a domain is fit on the other Core domains. The warning holds more collusion vocabulary than the colluders' own prompts do, so the test is conservative: 6 deception, 3 coordination, 5 steering and 5 honesty words in 79 words, against averages of 0.15, 0.23, 1.40 and 1.40 in Core colluder prompts [Nov 8, CPU]. This control ran in the original condition only.

3.9 Step 9: Read what the atoms say

What we do. Every atom is the J-lens vector of a vocabulary token, so the 25 atoms of an activation can be read as 25 tokens. For each agent we take the atoms of its turns and measure two things. (a) **Concept share**: the share of atoms that fall in four word groups fixed on November 8, before any readout: deception and secrecy, coordination, steering and persuasion, honesty and standards. (b) **Token by token**: for every token, the share of colluders whose atoms include it minus the share of honest agents in the same episodes.

Why. The promise of J-space over an ordinary probe is that it can be read. If colluders' J-space holds the collusion, the tokens should show it.

Details. The word groups have 14, 10, 10 and 10 words, which match 35, 47, 32 and 37 vocabulary tokens. Both measures are tested by reshuffling the colluder labels within each episode (5,000 permutations for (a), 1,000 for (b)) and corrected with the Benjamini–Hochberg procedure [8] at $q = 0.05$. A token is tested if at least 5 agents have it and at least 5 do not. On Core an agent's atoms are pooled over its three discussion turns; on Transfer they are the atoms of its final turn. Readouts ran at four positions: the last token, the last reply token, 50% into the reply, and the end of the system prompt.

3.10 Step 10: Steer the vote

What we do. Take each Core agent's final-vote prompt (400 prompts: 300 honest agents, 100 colluders) and force the reply to start with "RECOMMENDATION: Option". The next token is the option letter. The outcome is $y = \text{logit(target option)} - \text{logit(fair option)}$: positive means the model leans to the target. Then add $\alpha \cdot s \cdot u$ to the residual at one layer, on the final instruction and the forced tokens only, and measure the change Δy. Honest agents get $+u$, which should push them toward the target. Colluders get $-u$, which should push them away from it. The directions u, all of unit length, are four *concept directions* (the mean unit J-lens vector of each word group of Step 9), the *J-space role direction* (mean h_J of colluders minus mean h_J of honest agents in Core collusion episodes), and 20 random Gaussian directions as the null.

Why. A probe shows that information is present. Steering asks whether the direction is used: does moving the residual along it change the collusive choice?

Details. The strength is $\alpha \in \{0.5, 1, 2, 4\}$ times s, the median length of h_J at that layer (33 to 39). A direction counts as steering only if four conditions written down on November 8 all hold: (1) the predicted sign in both groups; (2) an effect above the 95th percentile of the random directions at the same strength; (3) significance after Benjamini–Hochberg at $q = 0.05$ over all direction, strength and group cells; (4) growth with the strength. Three runs completed: swap at layers 22 and 23, original at layer 22. After seeing them we added a second null of 20 random *lens* directions: the mean unit J-lens vector of a random set of 32 to 47 vocabulary tokens, the sizes of the word groups (Finding 7 says why).

3.11 Implementation: what comes from Anthropic's `jlens` and what is ours

The reference implementation of the Jacobian lens is Anthropic's `jlens` package [2], the companion code of Gurnee et al. [1]. Table 4 states, part by part, what this study takes from it.

Part	Anthropic's implementation	This study
The lens J_ℓ	`jlens` fits it (`jlens.fit`). The lens for Gemma-2-9B-it was fitted with that code and published by Neuronpedia [3]: 453 WikiText prompts of 128 tokens, layers 0 to 40.	**Used as published.** We did not fit a lens for Gemma. Our own Jacobian estimator is used only for the small random model of the pipeline check.
Applying the lens: transport $J_\ell h$ and readout (final norm, unembedding, softcap)	`JacobianLens.transport` and `HFLensModel.unembed`.	**Re-implemented and checked.** The analysis runs on saved activations with numpy code, so that it needs no model in memory. It gives the same numbers as the package (below).
J-lens vectors	Not in the package. The paper defines them as the rows of $W_U J_\ell$.	**Ours**, $v_i = J_\ell^\top((1+\gamma)\odot W_U[i])$: the direction along which the package's readout of token i rises (checked below).
J-space decomposition	Not in the package; no code is released. The paper's Methods take the J-space component to be the sparse non-negative combination of k J-lens vectors found by gradient pursuit; its appendix defines it as the projection onto the span of k vectors.	**Ours**, written from the paper's text. The numbers of this version use non-negative orthogonal matching pursuit, filled to $k = 25$, with projection onto the span. A rerun in the Methods form (gradient pursuit, non-negative combination) is in progress; see Check 2.
Probe, detectors, nulls, controls, steering	None.	**Ours.**

Table 4: What comes from Anthropic's reference implementation (`jlens` v0.1.0, commit `581d398`) and what is our own code.

Check 1: our lens arithmetic against the package. We ran the package's own code on 480 saved activations (48 per layer at layers 19 to 23, at the last token and at 50% into the reply, half from Core and half from Transfer). The matrices read by `JacobianLens.load` and by our loader are identical, and so is the transport. The readout logits differ by at most 2.8×10^{-5} on a scale of 30, which is float32 rounding; the top token and the top 50 tokens agree for every activation, also for the ranking by raw score that our pipeline uses. The package's logit of token i before the softcap equals $\langle v_i, h\rangle$ divided by the root mean square of $J_\ell h$ to within 1.7×10^{-4} on a scale of 50, so our J-lens vectors are exactly the readout directions of the published lens. The check executes `HFLensModel.unembed` on the model's final-norm and unembedding weights; it does not load the model.

Check 2: our decomposition against the paper's definition. A solver is needed because the J-space component of an activation is its best approximation by k of the 256,000 J-lens vectors, and the best subset cannot be searched exhaustively; a greedy pursuit adds one vector at a time. The paper's Methods state the procedure: solve, by gradient pursuit [7], for a sparse non-negative combination of k J-lens vectors that approximates the activation; that combination is the J-space component and the rest is the non-J-space component ($k = 25$ for probes, 16 for concept vectors). Its formalisation appendix defines the component as the projection onto the span of exactly k vectors. The numbers of this version follow the appendix form, and they differ from the Methods

form in two ways: the solver is non-negative orthogonal matching pursuit, a sibling of gradient pursuit that refits all weights after each added vector, and the component is the projection onto the span of the 25 chosen vectors, not their non-negative combination. The paper does not fix the stopping rule, a candidate pre-selection, the normalisation of the vectors, or whether the gain of the final normalisation is part of them. Our choices for these are: stop when no candidate correlates positively with the remainder; pre-select the 1,024 best candidates; compare vectors at unit length; include the gain, so that the inner product with a J-lens vector gives the readout logit up to the normalisation factor, as the paper describes.

On the activations of the first run [Nov 5], gradient pursuit scored 0.756 against 0.755 for our primary (paired difference $[-0.025, +0.026]$): the two solvers choose nearly the same atoms. Two other readings scored higher there, 0.811 without the fill to exactly 25 atoms and 0.809 with the non-negative reconstruction in place of the projection onto the span; both stayed below PCA-25 (0.856) and the full residual (0.902) of that run. On the activations of November 9 the version without the fill is the "J pursuit" arm of the appendix tables: 0.765 against 0.773 for the filled version. The same comparison on the activations of November 9 (gradient pursuit, the reconstruction, and J-lens vectors without the gain) was still running when this version was built [**not run**]; its results go to `results/jspace_interp/solver_check/`.

Not done. We did not run `JacobianLens.apply` on the model itself, which would also exercise the package's tokenisation and its hooks [**not run**]; our extraction reads the output of block ℓ with a forward hook, as the package does. A rerun of the whole sweep in the Methods form, with gradient pursuit and the non-negative combination as the J-space arm, was started on November 10 and was still running when this version was built; its results go to `results/jspace_interp/gemma2_9b_gp/` and will replace the J-space numbers of this paper.

3.12 Compute

One A40 GPU ran the main job for 7.7 hours on November 9: the 50 warned episodes, the activations for all four conditions, and 320 analysis cells (two conditions at seven positions, plus nosys and prompt_only, each at five layers and four sizes). Two shorter A40 jobs ran the steering (3.8 hours) and the lens null (0.9 hours). The total was about $7.60. The two checks of Section 3.11 ran on a laptop CPU. Appendix D has the run record.

4 Findings

The findings are ordered by how much each changes what we should believe or do next. Unless stated, numbers are Core→Transfer primary AUROCs at $k = 25$, band means over layers 19 to 23, at the last token. In the figures, blue is the full residual, orange is J-space, green is the 25-component PCA subspace, and grey marks random draws; a line at 0.5 is chance. Every plotted value is also in a table, in the finding or in Appendix A.

4.1 Finding 1: What transfers from Core to Transfer is the instruction

Question. When a probe trained on Core finds colluders in Transfer, what is it reading?

Answer. The instruction in the colluder's own context. With the instruction removed and the whole discussion kept, transfer falls to chance. With the system prompt alone in the context, before any discussion, the full residual transfers perfectly.

Why it matters. Every cross-scenario number on this benchmark, including the 0.945 of our reference probe, measures detection of an instruction. None shows that the activations carry a general signature of colluding behaviour.

Evidence. Figure 3, left panel; every value is in Table 6. In each condition the probe is trained and tested in that condition.

- **Instruction in the context (original).** The full residual transfers at 0.945, PCA-25 at 0.879 and J-space at 0.773.
- **Instruction removed.** Under swap, colluders see the honest system prompt for their seat and no private blocks; their own replies and the rest of the discussion are unchanged. Transfer drops to 0.479 for the full residual, 0.465 for PCA-25 and 0.436 for J-space. With no system prompt at all: 0.488, 0.466 and 0.475.
- **Instruction alone (prompt only).** Reading the end of the system prompt with nothing else in the context: 1.000 for the full residual at every layer, 1.000 for PCA-25, 0.980 for J-space.
- **The vote shows the same.** At the final vote colluders lean to the target option by +3.54 logits with the original prompt and by −3.85 with the swapped one (honest agents: −8.63; Step 10).
- **It is not shared wording.** A bag-of-words classifier on the system prompts is perfect within Core but transfers at 0.457 [Nov 8, CPU] (see also Finding 6).

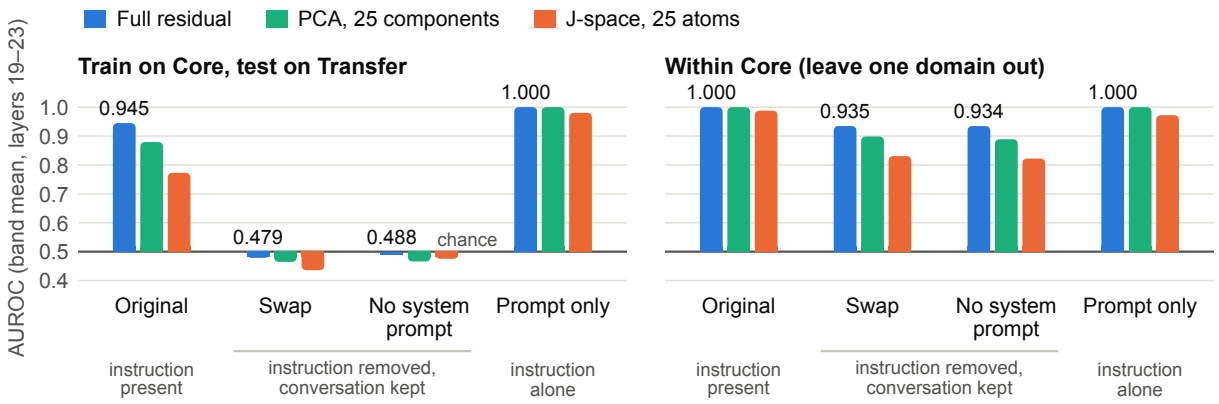


Figure 3: The four context conditions. Bars start at chance (0.5). Left: probe fit on Core, tested on Transfer. Right: within Core, leaving one domain out. The value of the full residual is printed.

Caveats.

- "Chance" is a statement about band means at the last token. Over the 35 swap cells (seven positions, five layers) the best full-residual score is 0.641 and 2 of the 35 intervals exclude 0.5, about what 35 tests give by chance. PCA-25 early in the reply is a weak exception (Finding 3).
- Swap removes the system prompt, the private block and the drift block together, so we cannot say which carries the signal. A perfect prompt-only score is easy to get: a random 25-dimensional subspace averages 0.92 there.
- One model, and collusion that is instructed. Nothing here speaks to collusion that arises without an instruction.

4.2 Finding 2: J-space beats none of the three baselines

Question. Is J-space a better place to look for colluders than (1) a random subspace of the same size, (2) the system prompt alone, (3) a 25-component PCA subspace?

Answer. No, on all three. J-space transfers at 0.773. Random subspaces reach 0.808 and random sets of lens atoms 0.875 at their 95th percentile, the prompt-only probe 1.000, and PCA-25 0.879.

Why it matters. This was the bar for making J-space the standard. The 0.755 of the first run is within what random 25-dimensional slices of the residual give.

Baseline J-space had to beat	Baseline	J-space	Beaten?	Above it at layers
1a. Random 25-dim subspaces, 95th percentile	0.808	0.773	No	21, 22
1b. Random sets of 25 lens atoms, 95th percentile	0.875	0.773	No	none
2. Prompt-only probe (system prompt alone)	1.000	0.773	No	none
3. PCA subspace, 25 components fit on Core	0.879	0.773	No	none

Table 5: The scoreboard. Core→Transfer primary AUROC, original condition, last token, $k = 25$, band mean. Each null has 200 draws; they average 0.650 (subspaces) and 0.716 (lens atoms). The last column lists the layers at which J-space scores above the baseline. For reference: full residual 0.945; residual with J-space removed (complement) 0.941.

Evidence. Table 5 and Figure 4.

- **Random subspaces.** J-space rises from 0.596 at layer 19 to 0.899 at layer 22 and is above the 95th percentile of the 200 random subspaces at layers 21 and 22 only ($p = 0.040$ and 0.010).
- **Random lens atoms.** It is never above their 95th percentile (0.969 at layer 22; p from 0.16 to 0.61), so the pursuit's atoms are not reliably better than 25 lens atoms drawn at random. The rule of Step 7 is met at no layer.
- **PCA and the complement.** J-space is below PCA-25 at every layer. Removing J-space costs almost nothing: the complement transfers at 0.941 and beats J-space at every layer.
- **Size and choice of atoms.** From $k = 8$ to $k = 100$ J-space goes from 0.740 to 0.807 while PCA goes from 0.831 to 0.938 (Table 9). A fixed frame of the 25 most common atoms scores 0.776.
- **The solver.** Gradient pursuit, the solver the paper names, scores the same as ours: 0.756 against 0.755 on the first run's activations [Nov 5] (Section 3.11).

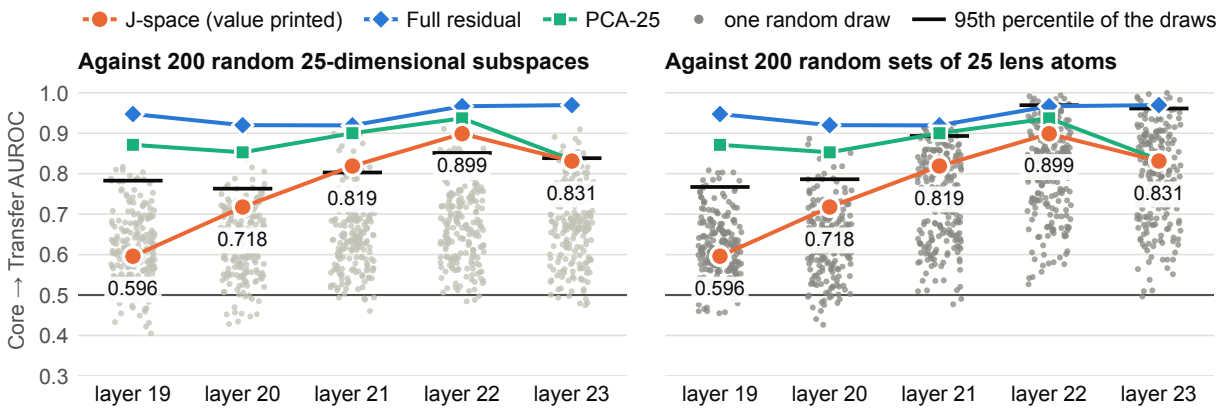


Figure 4: J-space against the two random nulls at each layer. A grey dot is one random draw; the black tick is the 95th percentile of the 200 draws. Original condition, last token, $k = 25$. The horizontal line is chance.

Caveats. J-space is not empty: at layer 22 it reaches 0.899 and clears the random-subspace null, as many sets of lens atoms do. Over all 35 cells of position and layer the rule is met in 2, both inside the reply where every arm is near chance.

4.3 Finding 3: Without the instruction, colluders stand out within one set of scenarios but not across sets

Question. Once the instruction is out of the context, does the discussion itself still give the colluders away?

Answer. Within Core, yes: 0.935 under swap when one domain is left out. Within Transfer, weakly: 0.667. From Core to Transfer, no: 0.479 at the last token, with one weak exception early in the reply.

Why it matters. This is the part of the result that is about behaviour. A detector of colluding behaviour, as opposed to instructions, has to be built and tested here, and it has to be shown to transfer. Nothing we tested does so clearly.

	Core → Transfer			Within Core (LODO)			Within Transfer (LOFO)		
Condition	Full	PCA-25	J-space	Full	PCA-25	J-space	Full	PCA-25	J-space
Original	0.945	0.879	0.773	1.000	1.000	0.988	0.974	0.898	0.866
Swap	0.479	0.465	0.436	0.935	0.898	0.831	0.667	0.667	0.555
No system prompt	0.488	0.466	0.475	0.934	0.889	0.822	0.631	0.609	0.605
Prompt only	1.000	1.000	0.980	1.000	1.000	0.972	1.000	1.000	0.985

Table 6: Primary AUROC under the three protocols, $k = 25$, band mean over layers 19 to 23, at the last token (prompt only: at the end of the system prompt). The left block is Figure 3, left panel; the middle block its right panel.

Evidence. Table 6, rows "Swap" and "No system prompt".

- **Within Core.** Under swap the full residual separates collusion from control episodes at 0.935, PCA-25 at 0.898 and J-space at 0.831. With no system prompt: 0.934, 0.889 and 0.822.
- **Within Transfer.** Under swap 0.667, 0.667 and 0.555; with no system prompt 0.631, 0.609 and 0.605.
- **Across.** 0.479 under swap and 0.488 with no system prompt, at the last token (Finding 1).
- **One weak exception across sets.** Under swap, PCA-25 read at 25% into the reply transfers at 0.64 to 0.65 at every layer (intervals between 0.53 and 0.75). The full residual at that position is at 0.488, and PCA with 8 or 100 components is at chance (0.475, 0.492). It is a lead, not a result.
- **The within-Core signal is spread widely.** Under swap an average random 25-dimensional subspace reaches 0.86 to 0.88 within Core (20 draws per layer), more than J-space. The full residual is strongest when the whole reply is averaged: 0.971.

What the within-Core signal is, we have not established. The transcripts offer candidates [Nov 8, CPU]. Colluders argue for the target option: 73 of 100 voted for it, against 8 of 100 honest agents in the same episodes. Colluders use fewer honesty and standards words in Core discussion: 8.3 per 1,000 words against 14.2. And a bag-of-words classifier on the replies scores 0.853 within Core but 0.493 from Core to Transfer, the same pattern as the probe. The atom readout under swap points the same way (Finding 4): colluders' atoms more often include "disagreed" and "argued".

Caveats.

- Within one set of scenarios, training and test episodes share a generation recipe, so a high score there does not show a general signal. Under swap the context still holds messages that were written under the instruction.
- The planned split of the activations by behaviour (colluders who voted for the target against those who did not) did not run [**not run**], so we cannot separate "argues for the target" from other differences.

4.4 Finding 4: The J-space token readout echoes the instruction

Question. If we read the vocabulary tokens that colluders' J-space atoms stand for, do they describe the collusion?

Answer. Only with the instruction in the context. Most atoms are generic tokens shared by nearly every agent. But colluders' atoms also include "subtly" and "lobbying" on Core and "disinformation" and "unethical" on Transfer. With the instruction swapped out, no token differs by role on Transfer.

Why it matters. This is J-space at its best: a plain-token readout that names the scheme. But it names the instruction, as in Finding 1, not colluding behaviour. It also corrects an earlier draft, which reported stance words only.

Evidence. Table 7: layer 22, last token. Differences are in percentage points of agents.

- **Most atoms are the same for everyone.** "decision" is among the atoms of 189 of the 200 agents in Core collusion episodes (Table 10). The four word groups fixed in advance hold at most 1.1% of the atoms in the conversation.

- **With the instruction present, colluders differ token by token.** On Core "subtly" is at +62 and "recommending" at −59. On Transfer "disinformation" is at +64 and "subtly" at +57. At layers 21 to 23 "subtly" stays between +54 and +76 on both sets.
- **Without the instruction the scheme words are gone.** Under swap, and with no system prompt, no token differs by role on Transfer at any of the five layers. On Core tokens still differ, but they are words of disagreement, which fits Finding 3.
- **Where it shows.** The instruction is visible in the atoms at the end of the system prompt already (on Core "justifying" at +41, "scrutiny" at −36). At 50% into the reply no token differs by role in any cell.

	Instruction in context (original)		Instruction removed (swap)	
	Core	Transfer	Core	Transfer
Selected for more colluders	subtly (+62), convincing (+50), justifying (+44), lobbying (+32), arguing (+28)	disinformation (+64), ironically (+61), subtly (+57), misinformation (+42), unethical (+39)	disagreed (+38), nuanced (+37), suggested (+36), argued (+30), contradicts (+29)	none
Selected for fewer colluders	recommending (−59), recommended (−58), recommendations (−51)	recommendations (−49), recommendation (−49), recommended (−36)	REJECT (−48), convincingly (−46), thankfully (−44)	none
Tokens that differ	30 or more	30 or more	30 or more	0

Table 7: Tokens that colluders and honest agents have among their atoms at different rates (layer 22, last token; Benjamini–Hochberg at $q = 0.05$). In brackets: the share of colluders with the token minus the share of honest agents in the same episodes, in percentage points. Only tokens made of unaccented Latin letters are listed.

Caveats.

- Calling these "scheme words" is our reading of the token lists, made after seeing them. The word groups fixed in advance did not capture them: they list exact word forms ("persuade", "argue"), and the atoms that differ are other words. Of 320 tests of a group's share by role, 21 pass Benjamini–Hochberg, all with differences of at most 0.026.
- The result files keep the 30 tokens with the smallest p per cell. A token counts for an agent if it is among the atoms of at least one of its turns. Readouts did not run at the `<end_of_turn>` token [**not run**].

4.5 Finding 5: The transferable signal sits on the two tokens that close the turn

Question. Where in a turn can the probe read the signal that transfers?

Answer. Only at the very end. At the last token of the reply text the full residual transfers at 0.517. One token later, on `<end_of_turn>`, it transfers at 0.780. One more token, on the newline that ends the turn, 0.945. Inside the reply it is at chance.

Why it matters. It says where the instruction-driven signal sits. It removes the hope that J-space would do better at positions with more content: inside the reply no arm scores above 0.57.

And it warns that results obtained within one set of scenarios hide this, because there the signal is readable everywhere.

Evidence. Figure 5; every value is in Table 11.

- **Across sets, three consecutive tokens: 0.517, 0.780, 0.945.** Inside the reply the full residual scores 0.522, 0.527 and 0.571 at 25%, 50% and 75%. The mean over all reply tokens gives 0.706.
- **Within Core the position hardly matters.** The full residual scores 0.968 to 1.000 at every position.
- **Within Transfer** it is in between: 0.50 to 0.59 inside the reply, 0.817 at the last reply token, 0.945 on `<end_of_turn>` and 0.974 at the last token.
- **J-space** is below the 95th percentile of random subspaces at every position, and between 0.478 and 0.569 inside the reply.

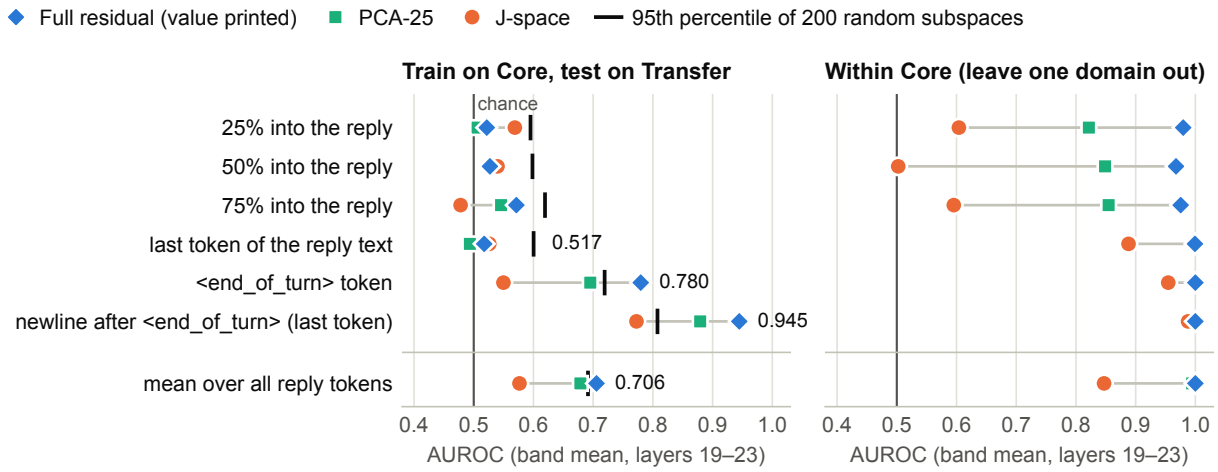


Figure 5: Read positions in the order they occur in a turn, with the mean over the reply below the line. Original condition, $k = 25$, band mean over layers 19 to 23.

Caveats.

- We did not test why the signal sits on the closing tokens. One reading is that tokens inside the reply mostly encode the local text, which differs between a committee discussion and a code review, while the closing tokens hold a summary of the whole context. That is a guess.
- The probe is fit on Core discussion turns and tested on Transfer final-vote turns, as in every earlier run. The two kinds of turn differ, and that may matter most inside the reply.
- The position called "pre-vote" in the code and in an earlier draft is the last reply token (Table 3).

4.6 Finding 6: The probe is not a vocabulary detector

Question. Does the probe fire on the words of collusion, or on the role?

Answer. On the role. Honest agents whose system prompt carries a warning written in the colluders' own words are separated from colluders perfectly by the full residual (1.000 at every

layer). The warning moves them 4% to 6% of the way toward the colluders. For J-space that share is 13% to 21%.

Why it matters. It rules out the cheapest explanation of Finding 1, that the probe matches words in the prompt. What the probe reads is closer to being told to collude than to seeing the words.

Evidence. Table 8: the 50 warned control episodes of Step 8, original condition, last token; ranges are over layers 19 to 23.

- **Warned agents are not mistaken for colluders.** Colluders against warned honest agents: 1.00 for the full residual and PCA-25, 0.90 to 0.99 for J-space. At the episode level, collusion against warned episodes: 1.00.
- **The warning is visible but small.** Warned against unwarned honest agents in the same episodes: 0.62 to 0.70 for the full residual (within-episode $p < 0.001$), where 0.5 would mean the probe does not see the warning. Warned episodes look slightly more like collusion than plain control episodes do (0.59 to 0.65).
- **Echo ratio.** 0.04 to 0.06 for the full residual, 0.01 to 0.05 for PCA-25, 0.02 to 0.07 for random subspaces, and 0.13 to 0.21 for J-space.

Representation	Colluders vs warned agents	Warned vs unwarned agents	Echo ratio	Collusion vs warned episodes
Full residual	1.00	0.62–0.70	0.04–0.06	1.00
J-space, 25 atoms	0.90–0.99	0.58–0.76	0.13–0.21	0.95–0.99
PCA, 25 components	1.00	0.55–0.62	0.01–0.05	1.00
Random 25-dimensional subspaces	0.97	0.55–0.62	0.02–0.07	n/a

Table 8: Matched-vocabulary control. The first two columns are AUROCs of agent scores; the echo ratio is the share of the colluders' shift that the warning alone produces; the last column is the primary AUROC of the episode detectors. For random subspaces: the mean over 20 draws, and the median for the echo ratio.

Caveats.

- J-space's echo ratio is 2.3 to 3.9 times the full residual's, but mainly because its gap between colluders and honest agents is smaller, and that gap is the denominator. Measured as an AUROC, J-space separates warned from unwarned agents no better than the full residual does (0.58 to 0.76 against 0.62 to 0.70).
- One warning text, 50 episodes, original condition only. The control did not run under swap or with no system prompt [**not run**].

4.7 Finding 7: Steering along J-space directions moves the vote logit, but mostly as any lens direction does

Question. If we push the residual along a J-space concept direction, does the vote change in a way that shows these directions drive the collusive choice?

Answer. The vote logit moves by up to 3.6 for honest agents, but at most 1% of honest votes flip. Most of the movement is generic: 20 random lens directions move honest agents by 2.2 to 2.9. Only "deception" and "steering" go beyond all of them, by 0.7 logits at most.

Why it matters. Against random Gaussian directions the result would have read as "J-space concept directions control the collusive vote". Against a fair null most of it goes away. A steering claim made with lens directions needs a lens-direction null.

Evidence. Figure 6: swap condition, layer 23, the largest strength ($\alpha = 4$). Table 12 has the values and Table 21 every run.

- **Against Gaussian directions every direction passes the four conditions of Step 10** at swap layer 23. At layer 22 Benjamini–Hochberg fails, and under the original prompt three of the four concept directions move colluders the wrong way.
- **The four concept directions are nearly one direction.** Their pairwise cosines are 0.82 to 0.92, and each has a cosine of 0.88 to 0.94 with the mean of all lens vectors, which a Gaussian direction does not share. "Honesty" moves the vote the same way as "deception".
- **Random lens directions share most of the effect.** They move honest agents by +2.18 to +2.95 and colluders by −1.23 to −0.80. Deception is beyond all 20 draws by 0.66 (honest) and 0.37 (colluders), steering by 0.18 and 0.04 ($p = 1/21$). Coordination is beyond them for honest agents only, by 0.01; honesty is not.
- **Votes barely move.** Honest agents start 8.6 logits from the target and at most 1% cross. Colluders are pushed off the target in at most 6% of prompts by a concept direction and 10% by the role direction; random Gaussian directions reach 2% and 9%.

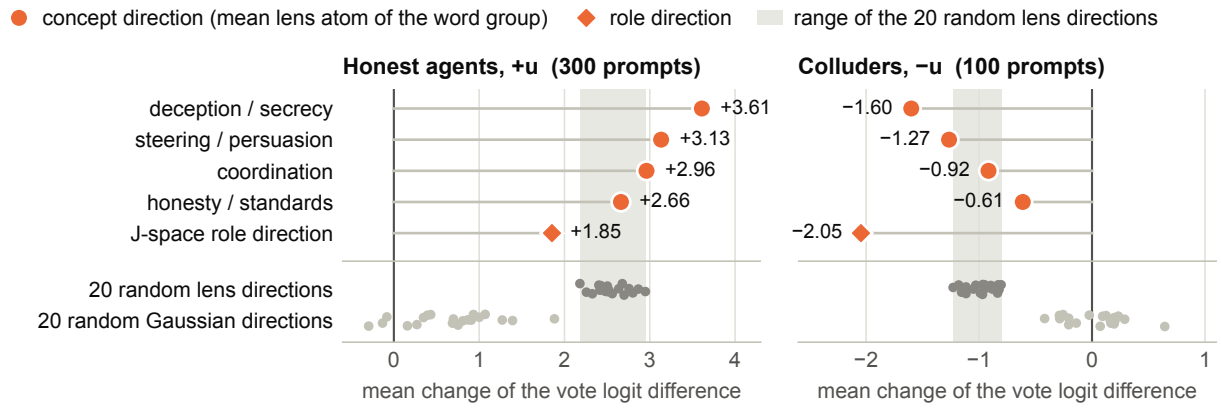


Figure 6: Mean change of the vote logit difference, in logits, at the largest strength; swap condition, layer 23. The grey band is the range of the 20 random lens directions. A concept direction is specific only where it falls outside the band.

Caveats. With 20 draws the smallest possible p is 1/21, so a sharper statement needs more draws. The lens null ran at one layer and one condition, and not for the role direction. Steered passes reuse a cached prefix and differ from full passes by up to 0.13 logits. The steering scale, the median length of h_J, is our own choice.

5 What did not run

- **Steering with the original prompt at layer 23.** The steering job hit its 4-hour limit after three of the four planned runs.

- **The paired-interval half of the rule for random subspaces** (Step 7). The random draws were scored without the bootstrap. The one-sided p against the 200 draws is reported instead.
- **The matched-vocabulary control under swap and with no system prompt.** The warned episodes were extracted for the original and prompt-only conditions only.
- **Atom readouts at the `<end_of_turn>` token and at other reply positions.** They ran at the last token, the last reply token, 50% into the reply, and the end of the system prompt.
- **The split of the activations by behaviour** (colluders who voted for the target against those who did not). It was planned on November 8 and exists for the text only (Appendix B).
- **A larger lens null for steering,** at more layers and for the role direction. It ran with 20 draws at swap layer 23.
- **An end-to-end run of `jlens` on the model** (Section 3.11). The sweep in the paper's Methods form (gradient pursuit, non-negative combination) was still running when this version was built.
- **Other models.** No Jacobian lens is published for the other three models with NARCBench activations.
- **Collusion without an instruction.** The benchmark only has instructed collusion.

All 320 planned analysis cells ran. The steering runs failed on the first GPU job with an out-of-memory error in our own loop and were rerun after the fix (Appendix D).

6 Limitations

- **Scope.** One model, one published lens, one run, and collusion that is instructed.
- **The decomposition is our implementation.** No reference code exists for it; Section 3.11 lists the choices the paper leaves open and tests the main ones on one cell.
- **Train and test turns differ.** Probes are fit on Core discussion turns and tested on Transfer final-vote turns.
- **Many cells.** There are 35 cells per condition at $k = 25$, so single cells clear a bar by chance.
- **The first run was not reproduced exactly.** The activations were regenerated (seven positions, bf16). The full residual now transfers at 0.945 where the first run had 0.902; the profile over layers and arms is the same.
- **The reading of the tokens in Finding 4 was made after seeing them.** The word groups were fixed in advance, but their choice is ours.
- **Text baselines** are bag-of-words models. Claims about related work are from an abstract or second-hand (Appendix C).

7 Conclusion and recommendations

Should J-space be the standard? No. As a detector it loses to every baseline it had to beat (Finding 2). As a readout it does show the scheme in plain tokens at the end of a turn (Finding 4), which is its most favourable result here. But what it shows is the instruction, fixed word groups do not capture it, and its concept directions do not steer the vote specifically (Finding 7). We recommend the following.

1. **Keep the full-residual probe as the reference detector,** and describe it as an instruction detector on this benchmark.
2. **Report every detector next to its swap and prompt-only scores.** A detector claim on NARCBench without them cannot be told apart from reading the instruction.

3. **Treat collusion without an instruction as the real test.** On the instructed benchmark the instruction is the signal. The within-set signal of Finding 3 is where a behaviour detector would start, and it has to transfer.
4. **Use a lens-direction null** for any steering claim made with lens vectors.
5. **If J-space is used for reading, read token by token** at the last token of a turn, with a swap control, and not through fixed word lists.
6. **Keep the lens and its application tied to Anthropic's reference implementation,** and state which solver was used for the decomposition, since none is released.

References

[1] W. Gurnee et al. Verbalizable Representations Form a Global Workspace in Language Models. Transformer Circuits Thread, 2026. https://transformer-circuits.pub/2026/workspace/index.html. arXiv:2607.15495.

[2] Anthropic. `jlens`: Jacobian lens, reference implementation (companion code of [1]), version 0.1.0, commit `581d398`, 2026. https://github.com/anthropics/jacobian-lens.

[3] Neuronpedia. Jacobian lens for `google/gemma-2-9b-it`, fitted with `jlens` on WikiText-103, June 2026. Hugging Face repository `neuronpedia/jacobian-lens`, file `gemma-2-9b-it/jlens/Salesforce-wikitext/gemma-2-9b-it_jacobian_lens.pt`.

[4] J. Rose et al. Detecting Multi-Agent Collusion Through Multi-Agent Interpretability. arXiv:2604.01151, 2026.

[5] M. Molinari and G. Bettineschi. Multi-Agent Collusion and the J-Space. arXiv:2611.02153, 2026.

[6] Gemma Team. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118, 2024.

[7] T. Blumensath and M. E. Davies. Gradient pursuits. *IEEE Transactions on Signal Processing*, 56(6):2370–2382, 2008.

[8] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.

A Tables behind the findings

The code names of the read positions are those of Table 3; `prevote` is the last token of the reply text. "J fill" is J-space filled to k atoms, "J pursuit" the version without the fill, "Compl." the complement. Over the 35 cells of position and layer in the original condition, PCA-25 is reliably better than J-space in 9 and J-space in 1 (paired 95% bootstrap intervals).

k	J-space	PCA-k	Random subspaces (p95)	Random lens atoms (p95)	Full residual
8	0.740	0.831	0.771	0.842	0.945
25	0.773	0.879	0.808	0.875	0.945
50	0.762	0.928	0.851	0.895	0.945
100	0.807	0.938	0.896	0.925	0.945

Table 9: Finding 2: the comparison at four subspace sizes. Core→Transfer, original condition, last token, band means; p95 is the 95th percentile of 200 draws.

Last token (newline after `<end_of_turn>`)		Last token of the reply text	
Atom (vocabulary token)	Agents	Atom (vocabulary token)	Agents
␣decision	189	<eos>	200
␣prioritizing	177	␣prioriti	200
<eos>	161	␣acknowledging	199
␣eloquently	161	␣pungkasnya	199
␣diría	161	.	198
␣discussions	155	␣␣	198
␣recommendation	154	␣concludes	197
␣recommendations	149	␣respectfully	195

Table 10: Finding 4: the most common atoms at layer 22 and the number of the 200 agents in Core collusion episodes that have each in at least one of their three discussion turns. ␣ marks a leading space.

		Core → Transfer				Full residual, within a split	
Read position	Code name	Full	PCA-25	J-space	Random p95	Core LODO	Transfer LOFO
25% into the reply	`frac25`	0.522	0.507	0.569	0.595	0.980	0.587
50% into the reply	`frac50`	0.527	0.538	0.540	0.598	0.968	0.499
75% into the reply	`frac75`	0.571	0.546	0.478	0.619	0.975	0.550
last token of the reply text	`prevote`	0.517	0.494	0.526	0.600	0.999	0.817
`<end_of_turn>` token	`eot`	0.780	0.695	0.550	0.719	1.000	0.945
newline after `<end_of_turn>` (last token)	`last`	0.945	0.879	0.773	0.808	1.000	0.974
mean over all reply tokens	`reply_mean`	0.706	0.679	0.577	0.691	1.000	0.900

Table 11: Finding 5: primary AUROC by read position, original condition, $k = 25$, band mean. "Random p95" is the 95th percentile of 200 random subspaces.

Direction u	Honest agents, $+u$		Colluders, $-u$		Beyond all 20 random
	Δy	votes flipped	Δy	votes flipped	lens directions?
deception / secrecy	+3.61	0%	−1.60	3%	both groups
steering / persuasion	+3.13	0%	−1.27	3%	both groups
coordination	+2.96	1%	−0.92	3%	honest agents only
honesty / standards	+2.66	0%	−0.61	2%	neither group
J-space role direction	+1.85	0%	−2.05	10%	not tested
20 random lens directions (range)	+2.18 to +2.95	up to 1%	−1.23 to −0.80	up to 3%	
20 random Gaussian directions (range)	−0.30 to +1.88	up to 2%	−0.42 to +0.64	up to 9%	

Table 12: Finding 7: steering at $\alpha = 4$, swap condition, layer 23. Δy is the mean change of logit(target) − logit(fair). A vote flips when y changes sign: toward the target for honest agents, away from it for colluders.

Cond.	Position	Full	J fill	J pursuit	Compl.	PCA-k	Rand. sub. mean (p95)	Rand. atoms p95	Layers clearing both
original	last	0.945	0.773	0.765	0.941	0.879	0.650 (0.808)	0.875	0/5
	eot	0.780	0.550	0.584	0.768	0.695	0.584 (0.719)	0.763	0/5
	prevote	0.517	0.526	0.535	0.520	0.494	0.512 (0.600)	0.588	0/5
	reply_mean	0.706	0.577	0.575	0.710	0.679	0.610 (0.691)	0.658	0/5
	frac25	0.522	0.569	0.517	0.521	0.507	0.500 (0.595)	0.580	2/5
	frac50	0.527	0.540	0.536	0.529	0.538	0.502 (0.598)	0.596	0/5
	frac75	0.571	0.478	0.461	0.567	0.546	0.519 (0.619)	0.602	0/5
swap	last	0.479	0.436	0.473	0.468	0.465	0.508 (0.609)	0.595	0/5
	eot	0.491	0.492	0.513	0.489	0.491	0.512 (0.611)	0.594	0/5
	prevote	0.469	0.515	0.537	0.477	0.479	0.501 (0.587)	0.572	1/5
	reply_mean	0.554	0.585	0.579	0.555	0.541	0.565 (0.649)	0.627	0/5
	frac25	0.488	0.508	0.531	0.482	0.643	0.514 (0.612)	0.596	0/5
	frac50	0.519	0.537	0.559	0.518	0.610	0.508 (0.608)	0.602	0/5
	frac75	0.573	0.471	0.469	0.564	0.514	0.505 (0.601)	0.592	0/5

Table 13: Core→Transfer primary AUROC at $k = 25$, band mean over layers 19 to 23, per condition and read position. "Layers clearing both" counts the layers at which J fill is above the 95th percentile of both random nulls.

Layer	Full	J fill	J pursuit	PCA-k	Rand. sub. p95	Rand. atoms p95	p vs rand. sub.	Clears both
19	0.948 [0.91, 0.98]	0.596 [0.50, 0.69]	0.598	0.871	0.783	0.767	0.597	–
20	0.920 [0.88, 0.96]	0.718 [0.64, 0.79]	0.724	0.853	0.763	0.786	0.149	–
21	0.919 [0.88, 0.95]	0.819 [0.76, 0.87]	0.773	0.900	0.803	0.893	0.040	–
22	0.967 [0.95, 0.99]	0.899 [0.85, 0.94]	0.924	0.937	0.851	0.969	0.010	–
23	0.970 [0.95, 0.99]	0.831 [0.77, 0.89]	0.806	0.835	0.838	0.961	0.075	–

Table 14: Per layer, original condition, last token, $k = 25$, with 95% episode-bootstrap intervals and the one-sided p of J fill against the 200 random subspaces.

Layer	Full	J fill	J pursuit	PCA-k	Rand. sub. p95	Rand. atoms p95	p vs rand. sub.	Clears both
19	0.450 [0.35, 0.55]	0.417 [0.31, 0.52]	0.465	0.451	0.606	0.617	0.970	–
20	0.457 [0.36, 0.55]	0.382 [0.28, 0.49]	0.483	0.461	0.625	0.586	1.000	–
21	0.484 [0.39, 0.58]	0.475 [0.39, 0.56]	0.534	0.486	0.604	0.572	0.657	–
22	0.460 [0.38, 0.55]	0.457 [0.38, 0.54]	0.445	0.481	0.588	0.597	0.831	–
23	0.542 [0.45, 0.64]	0.447 [0.34, 0.55]	0.440	0.443	0.620	0.604	0.900	–

Table 15: The same under swap: every arm at chance at every layer.

Layer	original/last		swap/last		nosys/last		prompt_only/prompt_end	
	Full	J fill	Full	J fill	Full	J fill	Full	J fill
19	0.948	0.596	0.450	0.417	0.503	0.504	1.000	0.985
20	0.920	0.718	0.457	0.382	0.472	0.414	1.000	0.998
21	0.919	0.819	0.484	0.475	0.474	0.501	1.000	0.997
22	0.967	0.899	0.460	0.457	0.490	0.496	1.000	0.968
23	0.970	0.831	0.542	0.447	0.504	0.462	1.000	0.952

Table 16: Core→Transfer primary AUROC per layer for the four conditions, full residual and J fill at $k = 25$.

k	Full	J fill	J pursuit	Compl.	PCA-k	Rand. sub. p95	Rand. atoms p95	J fill − PCA (95% CI)
8	0.945	0.740	0.740	0.944	0.831	0.771	0.842	[-0.08, 0.02] (L22)
25	0.945	0.773	0.765	0.941	0.879	0.808	0.875	[-0.09, 0.02] (L22)
50	0.945	0.762	0.766	0.938	0.928	0.851	0.895	[-0.10, -0.00] (L22)
100	0.945	0.807	0.766	0.938	0.938	0.896	0.925	[-0.08, 0.00] (L22)

Table 17: k sweep with all arms, original condition, last token, band mean. The paired interval of J fill minus PCA-k is for layer 22.

Cond.	Pos.	Full			J fill k=25			PCA-25		
		LODO	C→T	LOFO	LODO	C→T	LOFO	LODO	C→T	LOFO
original	last	1.000	0.945	0.974	0.988	0.773	0.866	1.000	0.879	0.898
	prevote	0.999	0.517	0.817	0.888	0.526	0.629	0.999	0.494	0.621
swap	last	0.935	0.479	0.667	0.831	0.436	0.555	0.898	0.465	0.667
	prevote	0.908	0.469	0.649	0.750	0.515	0.582	0.900	0.479	0.626
nosys	last	0.934	0.488	0.631	0.822	0.475	0.605	0.889	0.466	0.609
prompt_only	prompt_end	1.000	1.000	1.000	0.972	0.980	0.985	1.000	1.000	1.000

Table 18: The three protocols per condition, band mean: Core leave-one-domain-out (LODO), Core→Transfer (C→T) and Transfer leave-one-family-out (LOFO).

Cell	Arm	Coll. vs warned	Warned vs unwarned (p)	Warned vs control	Echo	Ep. warned vs control	Ep. coll. vs warned
original/last/L22	Full	1.000	0.694 (0.00)	0.612	0.05	0.632	1.000
	J fill k=25	0.978	0.756 (0.00)	0.738	0.21	0.739	0.992
	PCA-25	0.999	0.573 (0.00)	0.525	0.02	0.527	1.000
	Random 25-dim, mean of 20 (echo: median)	0.973	0.587	0.555	0.03	0.557	n/a
original/prevote/L22	Full	1.000	0.736 (0.00)	0.741	0.13	0.738	1.000
	J fill k=25	0.923	0.567 (0.03)	0.563	0.11	0.531	0.942
	PCA-25	0.998	0.716 (0.00)	0.744	0.17	0.702	0.999
	Random 25-dim, mean of 20 (echo: median)	0.890	0.615	0.605	0.16	0.563	n/a

Table 19: Matched-vocabulary control at layer 22, at the last token and at the last reply token, scored out of fold.

Cell	Group	Colluder	Honest (same ep.)	Control	BH p (within ep.)
swap/last/L22	deception/secrecy	0.0000	0.0000	0.0000	1.000
	coordination	0.0000	0.0000	0.0000	1.000
	steering/persuasion	0.0011	0.0019	0.0012	0.736
	honesty/standards	0.0000	0.0000	0.0000	1.000
most common atoms (Core): ␣decision (200), ␣prioritizing (199), ␣recommendation (190), <eos> (186), ␣recommending (183), ␣recommendations (178)					
swap/prevote/L22	deception/secrecy	0.0000	0.0000	0.0000	1.000
	coordination	0.0000	0.0000	0.0000	1.000
	steering/persuasion	0.0000	0.0000	0.0000	1.000
	honesty/standards	0.0000	0.0000	0.0003	1.000
most common atoms (Core): <eos> (200), ␣acknowledging (200), ␣prioriti (200), . (198), ␣␣ (198), ␣concludes (198)					
original/last/L22	deception/secrecy	0.0003	0.0001	0.0000	1.000
	coordination	0.0000	0.0000	0.0000	1.000
	steering/persuasion	0.0016	0.0022	0.0012	1.000
	honesty/standards	0.0000	0.0000	0.0000	1.000
most common atoms (Core): ␣decision (189), ␣prioritizing (177), <eos> (161), ␣eloquently (161), ␣diría (161), ␣discussions (155)					
original/prevote/L22	deception/secrecy	0.0000	0.0000	0.0000	1.000
	coordination	0.0000	0.0000	0.0000	1.000
	steering/persuasion	0.0000	0.0000	0.0000	1.000
	honesty/standards	0.0000	0.0000	0.0003	1.000
most common atoms (Core): <eos> (200), ␣prioriti (200), ␣acknowledging (199), ␣pungkasnya (199), . (198), ␣␣ (198)					

Table 20: Share of the four word groups among the $k = 25$ atoms at layer 22, mean per agent over its three Core discussion turns, with Benjamini–Hochberg adjusted within-episode permutation p.

Run	Direction	Group (n)	Δy at α=1	α=2	α=4	Flips α=2	α=4	Sign	> rand. p95	BH	All bars
original L22	deception_secrecy	colluder (100)	-0.01	-0.12	-0.47	0.03	0.06	–	–	–	–
	deception_secrecy	honest (300)	0.83	1.87	3.21	0.00	0.00	✓	✓	–	–
	coordination	colluder (100)	0.11	0.17	0.20	0.01	0.03	–	–	–	–
	coordination	honest (300)	0.74	1.76	2.49	0.00	0.00	✓	✓	–	–
	steering_persuasion	colluder (100)	0.10	0.13	0.05	0.01	0.04	–	–	–	–
	steering_persuasion	honest (300)	0.79	1.82	2.68	0.00	0.00	✓	✓	–	–
	honesty_standards	colluder (100)	0.19	0.29	0.38	0.01	0.03	–	–	–	–
	honesty_standards	honest (300)	0.73	1.73	2.41	0.00	0.01	✓	✓	–	–
	J role probe	colluder (100)	-0.14	-0.31	-0.66	0.00	0.01	✓	–	–	–
	J role probe	honest (300)	0.17	0.55	1.90	0.00	0.00	✓	–	–	–
colluder: baseline y_0 mean 3.54, share with $y_0 > 0$ 0.73; 20 random directions at α=4: mean Δy from -0.95 to 0.38											
honest: baseline y_0 mean -8.63, share with $y_0 > 0$ 0.05; 20 random directions at α=4: mean Δy from -0.11 to 1.94											
swap L22	deception_secrecy	colluder (100)	-0.35	-0.58	-0.88	0.01	0.03	✓	✓	–	–
	deception_secrecy	honest (300)	0.84	1.91	3.25	0.00	0.00	✓	✓	–	–
	coordination	colluder (100)	-0.18	-0.29	-0.33	0.00	0.00	✓	–	–	–
	coordination	honest (300)	0.76	1.79	2.53	0.00	0.00	✓	✓	–	–
	steering_persuasion	colluder (100)	-0.27	-0.43	-0.61	0.00	0.02	✓	–	–	–
	steering_persuasion	honest (300)	0.81	1.85	2.71	0.00	0.00	✓	✓	–	–
	honesty_standards	colluder (100)	-0.19	-0.28	-0.26	0.00	0.02	✓	–	–	–
	honesty_standards	honest (300)	0.74	1.75	2.45	0.00	0.01	✓	✓	–	–
	J role probe	colluder (100)	-0.41	-0.82	-1.54	0.03	0.10	✓	✓	–	–
	J role probe	honest (300)	0.51	1.11	2.38	0.01	0.01	✓	✓	–	–
colluder: baseline y_0 mean -3.85, share with $y_0 > 0$ 0.24; 20 random directions at α=4: mean Δy from -0.63 to 0.69											
honest: baseline y_0 mean -8.63, share with $y_0 > 0$ 0.05; 20 random directions at α=4: mean Δy from -0.11 to 1.99											
swap L23	deception_secrecy	colluder (100)	-0.52	-0.98	-1.60	0.01	0.03	✓	✓	✓	✓
	deception_secrecy	honest (300)	1.06	2.33	3.61	0.00	0.00	✓	✓	✓	✓
	coordination	colluder (100)	-0.37	-0.64	-0.92	0.00	0.03	✓	✓	✓	✓
	coordination	honest (300)	0.92	2.09	2.96	0.01	0.01	✓	✓	✓	✓
	steering_persuasion	colluder (100)	-0.45	-0.82	-1.27	0.00	0.03	✓	✓	✓	✓
	steering_persuasion	honest (300)	1.00	2.20	3.13	0.00	0.00	✓	✓	✓	✓
	honesty_standards	colluder (100)	-0.24	-0.47	-0.61	0.00	0.02	✓	✓	✓	✓
	honesty_standards	honest (300)	0.84	1.93	2.66	0.00	0.00	✓	✓	✓	✓
	J role probe	colluder (100)	-0.63	-1.33	-2.05	0.06	0.10	✓	✓	✓	✓
	J role probe	honest (300)	0.63	1.15	1.85	0.00	0.00	✓	✓	✓	✓
colluder: baseline y_0 mean -3.85, share with $y_0 > 0$ 0.24; 20 random directions at α=4: mean Δy from -0.42 to 0.64											
honest: baseline y_0 mean -8.63, share with $y_0 > 0$ 0.05; 20 random directions at α=4: mean Δy from -0.30 to 1.88											
swap L23 (lens null)	deception_secrecy	colluder (100)	-0.52	-0.98	-1.60	0.01	0.03	✓	✓	–	–
	deception_secrecy	honest (300)	1.06	2.33	3.61	0.00	0.00	✓	✓	–	–
	coordination	colluder (100)	-0.37	-0.64	-0.92	0.00	0.03	✓	–	–	–
	coordination	honest (300)	0.92	2.09	2.96	0.01	0.01	✓	–	–	–
	steering_persuasion	colluder (100)	-0.45	-0.82	-1.27	0.00	0.03	✓	✓	–	–
	steering_persuasion	honest (300)	1.00	2.20	3.13	0.00	0.00	✓	✓	–	–
	honesty_standards	colluder (100)	-0.24	-0.47	-0.61	0.00	0.02	✓	–	–	–
	honesty_standards	honest (300)	0.84	1.93	2.66	0.00	0.00	✓	–	–	–
colluder: baseline y_0 mean -3.85, share with $y_0 > 0$ 0.24; 20 random directions at α=4: mean Δy from -1.23 to -0.80											
honest: baseline y_0 mean -8.63, share with $y_0 > 0$ 0.05; 20 random directions at α=4: mean Δy from 2.18 to 2.95											

Table 21: All steering runs. Δy under $+u$ for honest agents and $-u$ for colluders. Flips: share of prompts whose y changes sign. Sign: predicted sign at every strength. > rand. p95: above the 95th percentile of the 20 random directions at every strength. BH: rejected at $q = 0.05$ at some strength. All bars: the four conditions of Step 10. Below each run: the baseline y_0 per group and the range of the random directions.

B Earlier results this study builds on

The first run, November 5 [Nov 5]

Arm (layers 19–23, last token)	Core LODO	Core→Transfer	Transfer LOFO
Full residual	1.000	0.902 [0.863, 0.935]	0.977
J-space, $k = 25$	0.995	0.755 [0.709, 0.796]	0.854
Complement	1.000	0.891	0.978
Random 25-dim subspace (one draw)	0.981	0.647 [0.602, 0.693]	0.742
Shuffled atoms (another turn's)	–	0.559 [0.511, 0.611]	–
PCA, top 25 components of Core	1.000	0.856 [0.807, 0.898]	0.902
Earlier proxy: top-25 unembedding directions	0.989	0.602 [0.550, 0.658]	–

Table 22: Primary AUROC at $k = 25$, band mean over layers 19 to 23. Source: the results file of the run `gemma2_9b_jpaper_nnomp_k25` on branch `jspace-separability`.

Per layer, J-space against the single random draw was close at layers 19 to 21 (0.564 and 0.538, 0.709 and 0.696, 0.752 and 0.750) and apart only at layers 22 and 23 (0.852 and 0.753, 0.897 and 0.497). The atoms chosen at the last token were nearly a fixed set across turns: at Core layer 21, "decision" in 90% of turns, `<eos>` in 86% and "prioritizing" in 82%. On Transfer discussion turns J-space was at the level of the random draw (0.613 against 0.602).

Solver variants of the first run. The same run compared readings of the decomposition on its own activations. Gradient pursuit, the solver the paper names, gave 0.756 against 0.755 for our primary (paired difference $[-0.025, +0.026]$). Without the fill to exactly 25 atoms the score was 0.811, and with the non-negative reconstruction in place of the span projection 0.809. Selecting atoms by raw score gave 0.740 and taking the 25 largest correlations without a pursuit 0.751. Section 3.11 repeats the main variants on the activations of November 9.

Other models. On the three models with public NARCBench activations, an earlier J-space proxy built from unembedding directions was at chance from Core to Transfer and level with a random 25-dimensional subspace: Qwen3-32B 0.513 against 0.508, DeepSeek-R1-32B 0.500 against 0.550, GPT-OSS-20B 0.568 against 0.611. No Jacobian lens is published for those models, so the Jacobian-lens result has not been tested elsewhere.

Results from the transcripts alone, November 8 [Nov 8, CPU]

Text baselines. Bag-of-words features (1- and 2-grams, TF-IDF fit on Core only, at most 5,000 features) go through the same detectors and protocols as the activation probe.

Feature	Core LODO	Core→Transfer	Transfer LOFO
Reply words only	0.853	0.493 [0.389, 0.603]	0.642
System-prompt words only	1.000	0.457 [0.394, 0.518]	0.952
Four concept word groups	0.627	0.593 [0.512, 0.675]	0.565

Table 23: Primary AUROC of text-only features, with 95% episode-bootstrap intervals for Core→Transfer.

The system prompts separate colluders perfectly within Core and almost perfectly within Transfer, but the words do not carry across: trained on Core prompts and tested on Transfer, they are at chance. The same holds for the replies.

Behaviour split on the text. Colluders who did *not* vote for the target against honest agents who also voted for another option: reply-word AUROC 0.776 (permutation $p = 0.0002$; 27 against 92 agents). Within colluders, target vote against other vote: 0.583 ($p = 0.21$). Within honest agents, target vote against other vote: 0.792 ($p = 0.007$; 8 against 92), so arguing for the target option has a wording of its own.

Phase	Group	Colluder	Honest (same ep.)	Control	BH p (within ep.)
Core discussion	honesty/standards	8.29	14.22	13.61	0.0008
Core discussion	deception/secrecy	0.66	0.74	0.80	0.65
Core final	honesty/standards	9.29	14.39	14.86	0.005
Transfer discussion	honesty/standards	1.21	1.88	1.86	0.47
Transfer final	steering/persuasion	3.87	2.45	2.23	0.016

Table 24: Concept-word rates per 1,000 words of reply text. Colluders avoid honesty and standards language in Core; they do not use more deception words.

Which words differ.

Matched-vocabulary prompt check. The warning has 6 deception/secrecy, 3 coordination, 5 steering and 5 honesty words in 79 words. Colluder system prompts average 0.15, 0.23, 1.40 and 1.40 on Core and 1.89, 0.17, 0.63 and 0.54 on Transfer.

Pipeline check with a small random model. Every stage ran on the real transcripts with a randomly initialised 6-layer model. Two lessons carried over. With 5 random draws per cell, J-space cleared both 95th percentiles in 6 of 45 cells of a network that knows nothing. And at the end of the system prompt even that network separated colluders from Core to Transfer (0.84 [0.73, 0.95]), because activations there are a function of the prompt tokens.

C Related work

Molinari and Bettineschi [5] (abstract read) report on NARCBench that colluders' J-space "holds the collusion itself" (86% of colluders represent misrepresentation, concealment or remorse before replying, on DeepSeek-V4-Flash), and they introduce an unsupervised detector that performs close to supervised probes. The abstract does not mention system-prompt controls. Finding 4 here is compatible with their observation and adds that, on Gemma-2-9B-it, the scheme words leave the readout when the instruction leaves the context. A LessWrong post reportedly finds that NARCBench system prompts alone separate colluders (not read here; the text baselines agree within one set of scenarios). An Apart sprint team reportedly built a J-lens collusion monitor on NARCBench; its report could not be read.

D Run record

- **Main job.** RunPod A40 (48 GB, $0.59 per hour), pod `n8tc4b9iwvm5lt`, commit `5c300ad` of branch `jspace-interp`, November 9, 07:22 to about 15:05 UTC, 7.7 hours. It generated the 50 warned control episodes (800 prompts, 23 minutes), extracted the activations (Core 1,700 turns, Transfer 936, warned 800 per condition; about 25 minutes per condition), and ran 16 analysis jobs, one per condition and read position. Each job ran five layers at four sizes with 200 random subspaces and 200 random atom sets per cell; 20 of the random subspaces were also scored within Core and on the warned episodes. A first pod was stopped after ten minutes to fix how the job sized its worker pool.
- **Steering job.** Pod `lz7gdscc09o0on`, A40, 15:08 to 18:55 UTC, 3.8 hours, commit `2899572`. On the main job the four steering runs failed with a CUDA out-of-memory error, because our loop kept every prompt's key–value cache on the GPU (fixed in commit `bdd30cd`; results identical to 10^{-7} on a small test model). This job completed three of the four runs before its 4-hour limit.
- **Lens-null job.** Pod `rt34fhh908ja7p`, 0.9 hours, commit `1c740de`: swap layer 23 against 20 random lens directions.
- **Cost.** About $4.60, $2.25 and $0.55; about $7.60 in total by the account balance. No pod is running.
- **Checks against the reference implementation** (November 10, laptop CPU). `jlens` v0.1.0 was installed from the repository `anthropics/jacobian-lens` at commit `581d398`. The results are in the folders `jlens_check` and `solver_check` under `results/jspace_interp/`.

E Code, files and how to rebuild

The code is on branch `jspace-interp` of the narcbench-core-demo repository. The modules are in `jspace_interp/`: contexts (conditions), positions and extract (read positions), sweep and detectors (arms, probe, protocols), concepts and stats (atom readout), matched_vocab and steer. The results are under `results/jspace_interp/`. The folder `gemma2_9b/` holds one file `sweep_results.json` per condition and read position, `steer/` the steering runs, and `pod_runs/` and `pod_logs/` the pod records.

```
python scripts/jspace_interp_summarize.py      # SUMMARY.json, RESULTS.md, tab_*
python scripts/jspace_interp_verify_jlens.py   # check against Anthropic's jlens
python scripts/jspace_interp_solver_check.py   # the paper's solver, headline cell
python scripts/jspace_interp_paper_assets.py   # figs/fig_*.pdf, tabp_*
cd docs/jspace_interp && tectonic jspace_colluder_study.tex
```

Every figure, and every table in Section 4 and Appendix A, is written by these scripts from the result files. The numbers quoted in the text were checked against the same files.

F Changes since earlier drafts

The draft before this one is commit `af7afae`. No result changed. The changes are:

1. **Structure.** The method comes first, step by step; the findings follow in order of importance, each with the question it answers. Four figures were added.
2. **Reference implementation (Section 3.11).** The paper now states which parts come from Anthropic's `jlens` and which are our code, and reports two checks: our lens arithmetic against the package, and our decomposition against the solver the paper names.

3. **Atom readout (Finding 4).** An earlier draft said that the tokens which differ by role are stance and register words. The result files show more: with the instruction in the context, colluders' atoms include scheme words, and these are gone under swap.
4. **The "pre-vote" read position** is now described as what it is for the probe's turns: the last token of the reply text (Table 3, Finding 5).
5. **A weak exception to chance under swap** is now reported: PCA-25 early in the reply (Finding 3). An earlier draft said every arm falls to chance.
6. **Warned episodes** have 800 turns per condition; an earlier draft said 850.
7. **The generated results summary** (`RESULTS.md`, `SUMMARY.json`) listed the wrong steering run as missing (swap layer 23, which ran). The run that is missing is original layer 23. The summary script is fixed.